Coordination by hand is where disaster response loses people

DispatchProof · AgentX PS-03 · Team “Ratnesh's team” (CB9B1D)

- **The problem, in the organisers' words:** during floods and heatwaves, alerts, rescue requests and relief supplies are coordinated by hand.
- **Reports arrive duplicated, in two languages, with facts missing —** and a plan approved against yesterday's crew availability gets executed today.
- **DispatchProof:** two cooperating agents, one code-enforced approval gate, one append-only audit trail.
- **Ward alerts and area-wise updates in Hindi and English —** the PS-03 brief, covered: a synthetic alert is correlated by Agent A and reflected in the ward situation update.
- **The output a duty officer can trust:** an acknowledged, evidence-backed responder assignment — not a message that says “rescued”.

All data in this deck and demo is synthetic: the fictional ward “Ghatauli”, scripted reports, fictional crews.

Two agents, a hard boundary, one gate

The separation is in the code, not in the prompts

Agent A — Intake & Verification
extracts facts · links duplicates · flags missing info
NO code path to dispatch

Agent B — Resource Coordination
matches capability → best-fit capacity → proximity
reserves crews · cannot approve or dispatch
severity is read-only

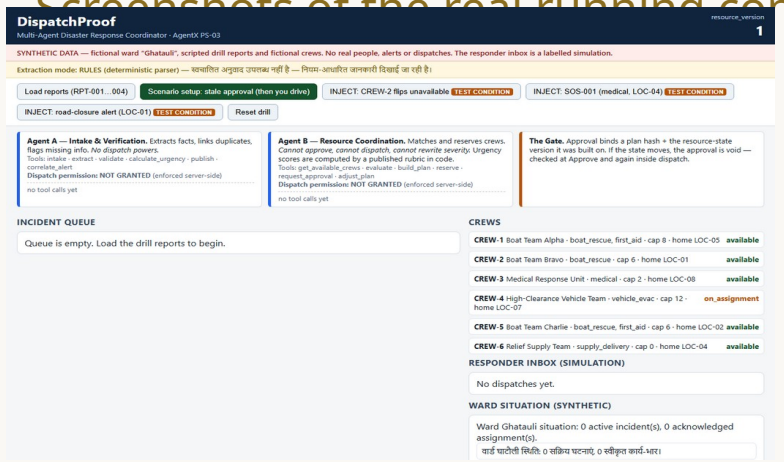
The Gate + tools.py
approval binds plan hash + resource-state version
checked at Approve AND inside dispatch
dispatch exists only as a tool call
Dispatch permission for both agents: NOT GRANTED
(enforced server-side — an agent identity gets 403)

- **Shared memory:** one SQLite case store — reports, incidents, plans, approvals, dispatches, acknowledgements, audit. State survives refresh and restart.
- **Scoring is code:** the urgency rubric is a published table computed by a pure function. The model layer only extracts facts, and the system runs in a labelled rules mode without it.

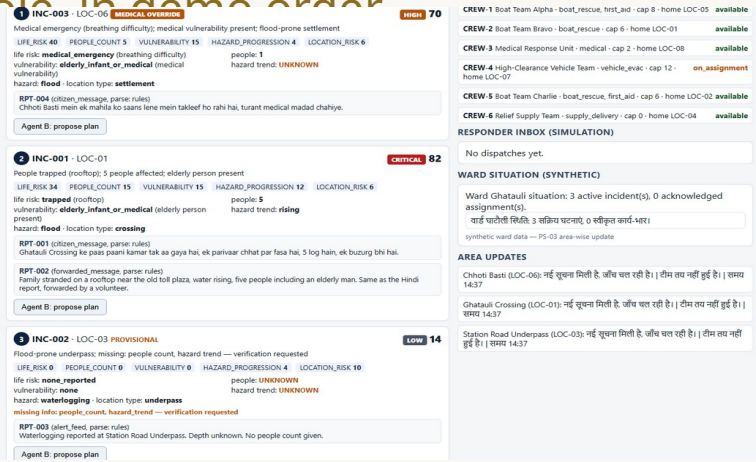
Import graph: agent_a → {store, rubric, llm} · agent_b → {gate, rubric} · tools → {gate, store, templates_hi, fixtures} · neither agent imports tools.

The demo is the evidence

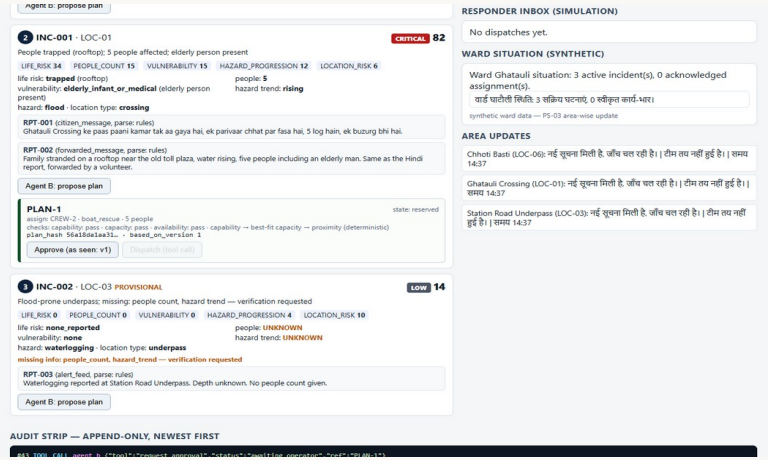
Screenshots of the real running console, in demo order.



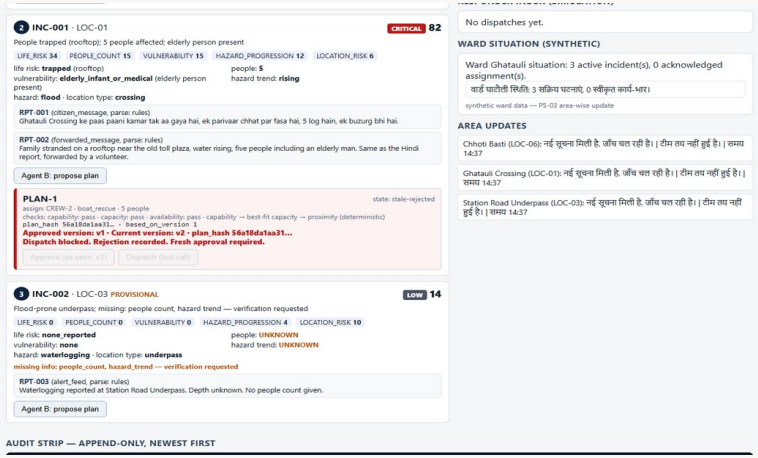
1 · Empty console, synthetic banner, audit at zero



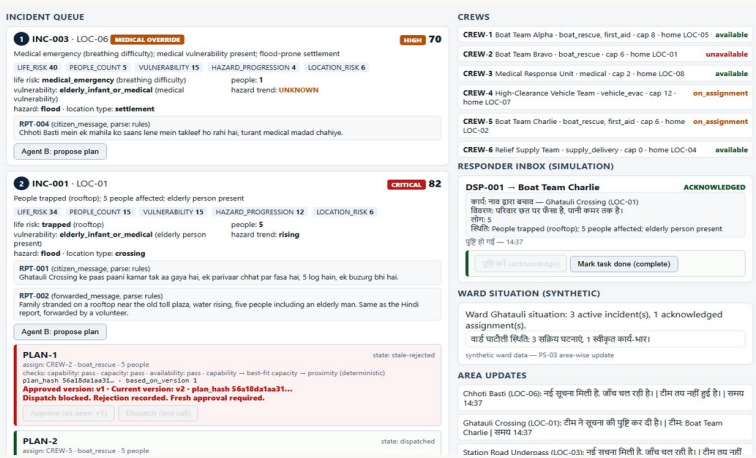
2 · Duplicate pair linked — both texts, one incident



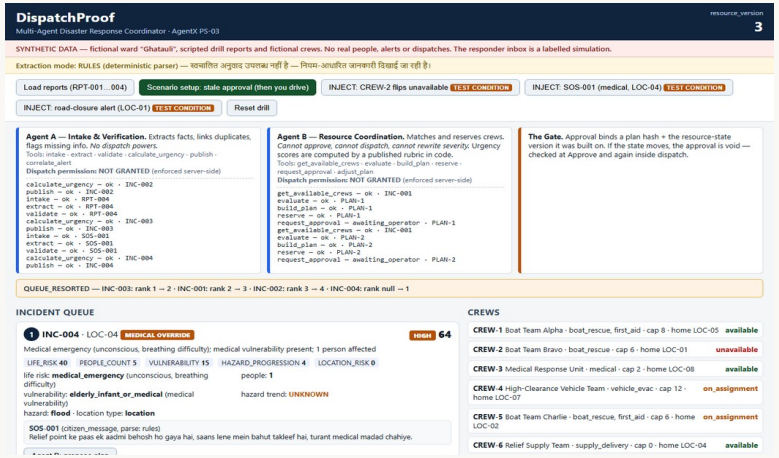
3 · Plan card: hash + resource version



4 · REJECTED_STALE — the stale approval refused



5 · Responder inbox: Hindi task, acknowledged



6 · Medical SOS takes queue rank 1

Declared before the build, run cold

Ten predeclared cases · run 2026-11-11 against the built product · pytest, one command

T01	Stale approval is rejected (409, audit order)	PASS
T02	Revised plan approves; approval is single-use	PASS
T03	Dispatch → acknowledge → persistence after refresh	PASS
T04	Duplicate reports link, never discard (both orders)	PASS
T05	Rubric scoring exact: 82 / 14 / 70, reproducible	PASS
T06	Audit completeness + append-only enforcement	PASS
T07	Medical SOS jumps the queue (rank 1)	PASS
T08	Restart recovery — state intact from SQLite	PASS
T09	Rules-mode fallback, labelled per report	PASS
T10	Agent separation is structural (import graph)	PASS

Result: 18/18 PASS — the 10 predeclared cases above plus 8 review-added cases (server-side authorization, idempotency, concurrency, committed rejection, alert correlation, tool traces, 2 Round-1 regressions), reported separately in tests/test_deltas.py. Judges can re-run: python -m pytest

What this system does not do

Honesty and limits — named before a judge finds them

- **All data is synthetic:** the fictional ward Ghatauli, scripted reports, fictional crews — labelled on every screen.
- **The responder inbox is a labelled simulation:** no real SMS or WhatsApp is sent to anyone.
- **The crew flip is an injected test condition,** disclosed here, on screen, and in the audit payload.
- **One ward, one scenario (flood).** No maps, no real alert feeds, no multi-ward scaling claims.
- **Extraction runs in rules mode in this build:** a deterministic parser; the model adapter is wired but no key is configured, and every audit row says so.
- **The audit is tamper-evident, not tamper-proof —** and the claim is bounded: stale-plan dispatch is prevented under the tested assumptions. Nothing here claims to make emergency response safe.

Consistency between deck, video, README and repo is the credibility play — every claim here is checkable in the code.

Stated targets, not measured results

What a field deployment would be held to

- **Target:** every approved dispatch bound to the state it was approved against — zero stale executions, enforced by the gate.
- **Target:** acknowledgement-separated dispatch states for 100% of assignments, each reconstructible from the audit alone.
- **Target:** duplicate reports linked without data loss, in Hindi and English intake alike.
- **A pilot needs next, in this order:** real alert-feed integration, responder identity, multi-ward state.
-
- **We did not mark any incident “rescued”.** We established an acknowledged assignment, with evidence of what happened.